

PL/SQL Training

SESSION 1: GETTING STARTED WITH PL/SQL & DECLARING PL/SQL VARIABLES

DAVA.X ACADEMY SESSIONS

Svetlana Sura & Iulian Aurel Cozma

10–20 JUNE 2025

SESSION 1

GETTING STARTED WITH PL/SQL

- Introduction to PL/SQL
- Features of PL/SQL
- Differences between SQL and PL/SQL
- PL/SQL architecture
- Different type of PL/SQL units
- PL/SQL Engine
- PL/SQL Block
- The PL/SQL Comments



Introduction to PL/SQL

- PL/SQL stands for “Procedural Language extensions to the structured Query Language
- The PL/SQL programming language was developed by Oracle Corporation in the late 1980s as procedural extension language for SQL and the Oracle relational database.
- Following are certain notable facts about PL/SQL :
 - PL/SQL is a completely portable, high-performance transaction-processing language.
 - PL/SQL provides a built-in, interpreted and OS independent programming environment.
 - PL/SQL's general syntax is based on that of ADA and Pascal programming language.
 - Apart from Oracle, PL/SQL is available in TimesTen in-memory database and IBM DB2.

Features of PL/SQL

Portable:	PL/SQL applications can be executed with all types of operating system where we have oracle installed.
Efficient:	All sorts of calculations can be efficiently performed by PL/SQL without the use of oracle engine. This improves transaction performance.
Error-checking	PL/SQL allows error-checking and displays user-friendly messages when error occurs.
Exception Handling:	PL/SQL code is capable of handling exceptions that can affect the flow of program, hence helps in making the code more reliable

Categories of SQL Operations

Category	Name	Purpose	Examples
DML	Data Manipulation Language	Used to manipulate data within existing tables	<u>Used to perform CRUD operations on data:</u> INSERT: Add new records UPDATE: Modify existing records DELETE: Remove records SELECT: Retrieve records (read-only, sometimes considered part of DQL)
DDL	Data Definition Language	Defines and manages database schema (structure)	<u>Used to define or alter database structure:</u> CREATE: Create new tables, views, indexes, etc. ALTER: Modify existing database objects DROP: Delete objects permanently TRUNCATE: Quickly delete all records from a table (faster than DELETE)
DCL	Data Control Language	Controls access to data in the database	<u>Used to control user permissions:</u> GRANT: Allow specific users to perform actions REVOKE: Remove previously granted permissions
TCL	Transaction Control Language	Manages changes made by DML statements	Used to manage transactions (groups of SQL operations that should succeed or fail together): COMMIT: Save all changes ROLLBACK: Undo changes since last COMMIT SAVEPOINT: Set a point to which you can later roll back

Differences between SQL and PL/SQL

SQL	PL/SQL
SQL is a single query that is used to perform DML and DDL operations, etc.	PL/SQL is a block of codes that used to write the entire program blocks/ procedure/ function, etc.
It is declarative, that defines what needs to be done, rather than how things need to be done.	PL/SQL is procedural that defines how the things needs to be done.
Execute as a single statement.	Execute as a whole block.
Mainly used to manipulate data.	Mainly used to create an application.
Cannot contain PL/SQL code in it.	It is an extension of SQL, so it can contain SQL inside it.



PL/SQL Architecture Components

1. PL/SQL Engine

- **Core component** responsible for processing PL/SQL blocks.
- It separates **procedural statements** (handled by the engine) and **SQL statements** (sent to the SQL engine).
- Can reside in:
 - **Oracle server** (most common)
 - **Oracle tools** like Forms or Reports (for client-side execution)

2. SQL Engine

- Handles **SQL statements** (e.g., SELECT, INSERT, UPDATE, DELETE).
- Performs **parsing, optimization, and execution** of SQL commands.
- Receives SQL from the PL/SQL engine and executes it in the database.

3. PL/SQL Block

- The basic unit of PL/SQL, processed by the PL/SQL engine.
- Divided into sections.

4. Database Server

- The actual Oracle RDBMS where data is stored and SQL is executed.
- Communicates with both PL/SQL and SQL engines.

PL/SQL Engine



PL/SQL engine is the component where the actual processing of the codes takes place.



PL/SQL engine separates PL/SQL units and SQL part in the input.



The separated PL/SQL units will be handled by the PL/SQL engine itself.



The SQL part will be sent to database server where the actual interaction with database takes place.



It can be installed in both database server and in the application server

PL/SQL Engine

- How PL/SQL Execution works:

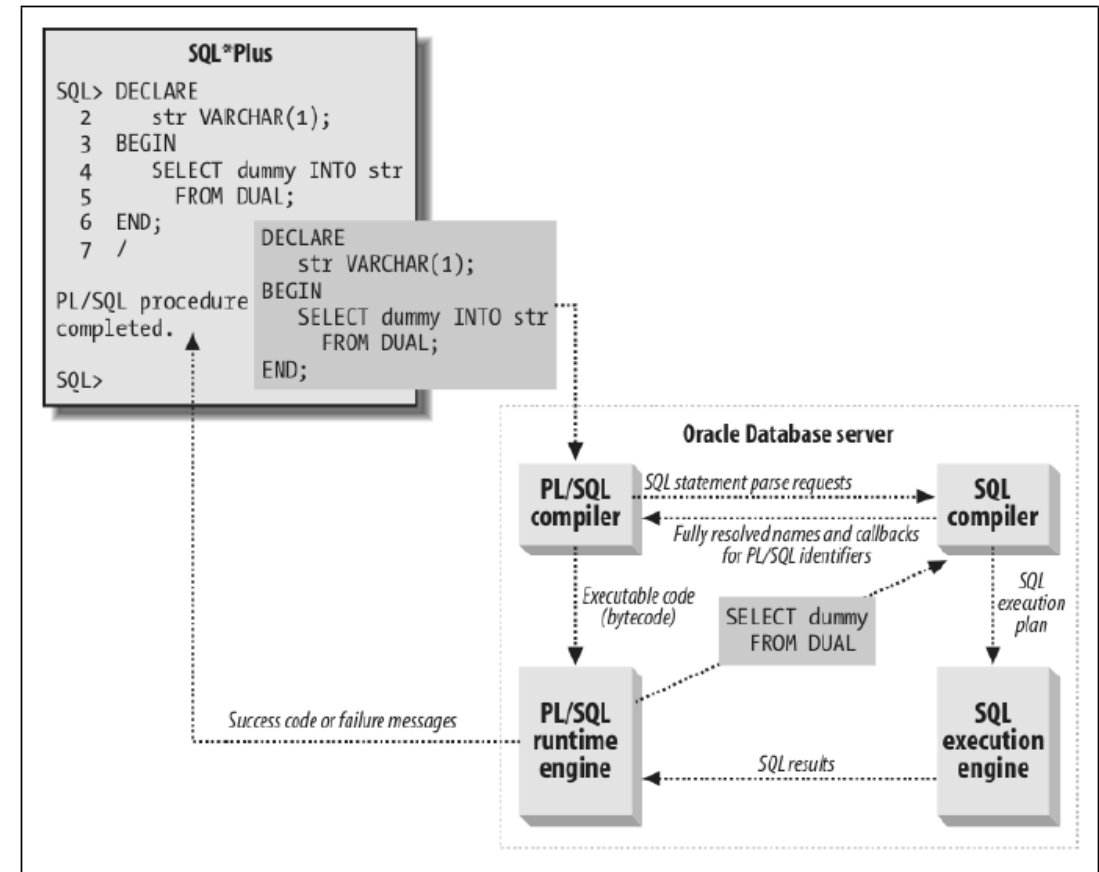
1. Code is sent to PL/SQL compiler which checks if the code adheres to the language's syntax;
2. Communicates with the SQL compiler to hand off any embedded SQL for execution;
3. The SQL parser will resolve any expressions, look for opportunities to use the function result cache (starting with Oracle Database 11g), execute semantic and syntax checks, perform name resolution and determine an optimal execution plan.
4. When a PL/SQL program is compiled, embedded SQL statements are modified slightly such that INTO clauses are removed, local program variables are replaced with bind variables, and some keywords are forced to uppercase.

- - For example, if myvar is a local program variable, PL/SQL will change this:

select dummy into str from dual where dummy = myvar;

into something like this:

SELECT DUMMY FROM DUAL WHERE DUMMY = :B1;
- 5. To the prepared SQL statement is applied any substitution of bind variables;
- 6. The execution and fetch of the SQL statement.

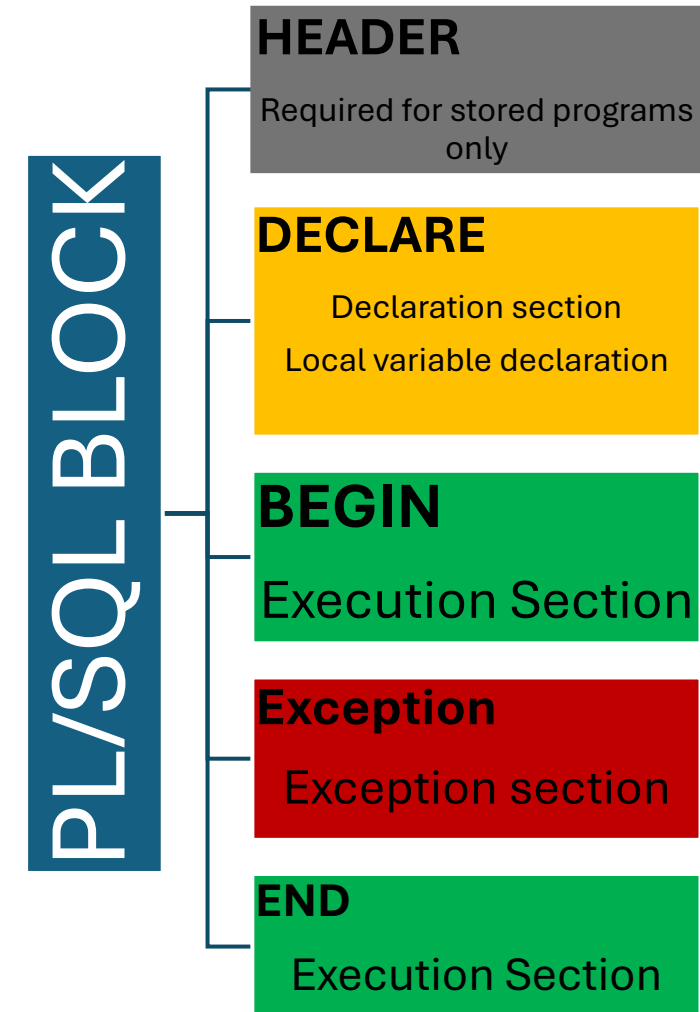


PL/SQL Block

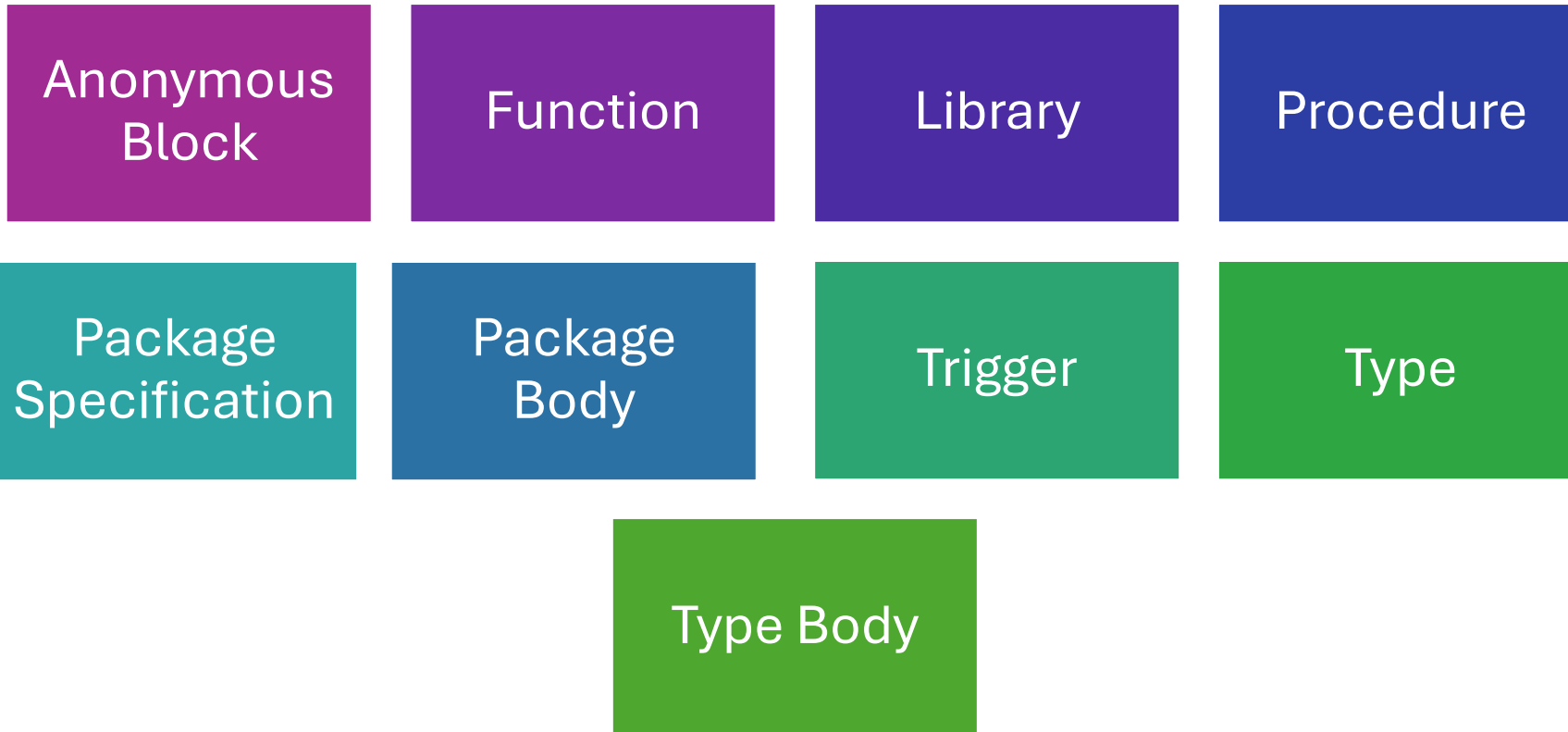
This is the component which has the actual PL/SQL code.

The skeleton of a PL/SQL block has four sections:

- **Header:** This is an optional section which is required for the named blocks. It contains block name, block owner's name, parameter specifications, and return type specification (for functions).
- **Declaration:** This is an optional section which is used for declaration of local variables, cursors, and local subprograms. The **DECLARE** keyword indicates the beginning of the declaration section.
- **Execution:** This is the mandatory section of a PL/SQL block which contains the executable statements. These statements are parsed by the PL/SQL engine and executed on the block invocation.
 - Execution section starts with **BEGIN** and ends with **END** keyword.
 - This is a mandatory section and here the program logic is written to perform any task like loops and conditional statements.
 - It should have at least one executable line of code, which may be just a NULL command to indicate that nothing should be executed.
- **Exception:** This is the optional section of the block which contains the exception handlers. The appropriate exception handler is activated upon any exception raised from the executable section to suggest alternate steps. The **EXCEPTION** keyword indicates the start of the exception section.



Following are the different type of PL/SQL units



PL/SQL anonymous block example

The following example shows a simple PL/SQL anonymous block with one executable section.

To run the code from the SQL command line, you may need to type / at the beginning of the first blank line after the last line of the code.

This is the component which has the actual PL/SQL code.

```
SET SERVEROUTPUT ON  
BEGIN  
DBMS_OUTPUT.PUT_LINE('Hello There ');  
END;  
/  

```

When the above code is executed at the SQL prompt, it produces the following result:

```
Hello There  
PL/SQL procedure successfully
```

Whenever you start Oracle SQL (PL/SQL) at that time you must have to write the "SET Serveroutput ON" command.

PL/SQL program execution into Oracle engine so we always required to get serveroutput result and display into the screen otherwise result can't be display.



PL/SQL Execution Flow

PL/SQL Code Submission

Code is sent to Oracle where it's compiled by the **PL/SQL compiler**.

Syntax & Semantic Checks

The PL/SQL engine checks the syntax of procedural code.

The SQL engine handles embedded SQL checks.

SQL Delegation

SQL inside PL/SQL is passed to the SQL engine.

Procedural logic is executed by the PL/SQL engine.

Execution

Bind variables are resolved.

SQL is executed, and results returned to PL/SQL for further logic or output.

The PL/SQL Comments

The PL/SQL supports single-line and multi-line comments.

All characters available inside any comment are ignored by the PL/SQL compiler.

The PL/SQL single-line comments start with the delimiter -- (double hyphen) and multi-line comments are enclosed by /* and */.

Syntax Using -- symbol	Syntax Using /* and */ symbols
--comment goes here	/* comment goes here */

Syntax Using Comment on Multiple Lines is:

```
/*  
* Author: Company User  
* Description: Get employee list  
* Creation date: June 2025  
*  
*/
```

SESSION 1 :

DECLARING PL/SQL VARIABLES

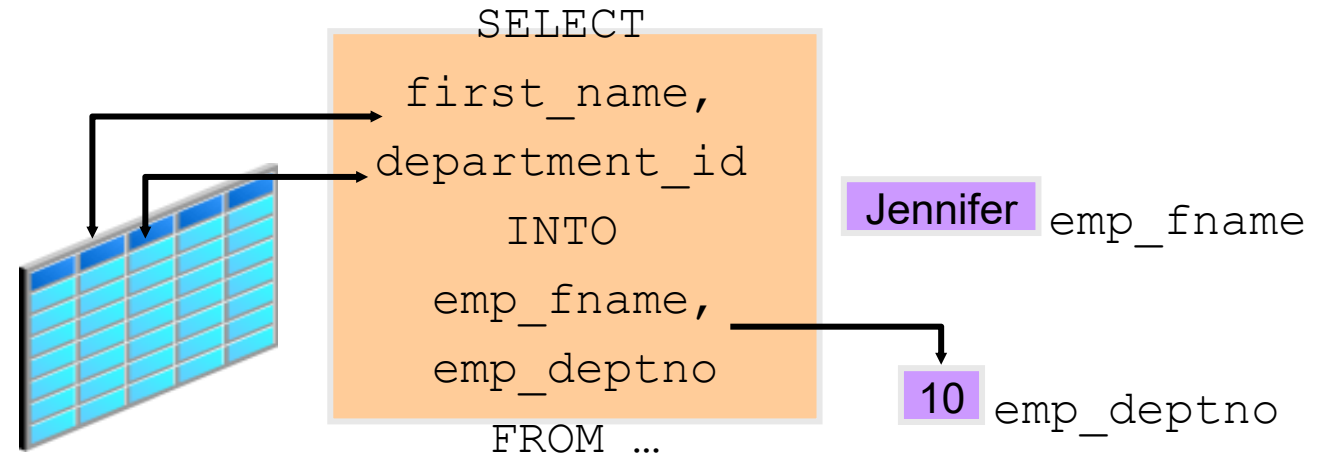
- Identify valid and invalid identifiers
- List the uses of variables
- Declare and initialize variables
- List and describe various data types
- Identify the benefits of using %TYPE attribute
-



PL/SQL Variables

Variables can be used for the following:

- Temporary storage of data
- Manipulation of stored values
- Reusability



A variable is a meaningful name which facilitates a programmer to store data temporarily during the execution of code. It helps you to manipulate data in PL/SQL programs. It is nothing except a name given to a storage area. Each variable in the PL/SQL has a specific data type which defines the size and layout of the variable's memory.

Naming rules for PL/SQL variables

- Identifiers are used for Naming a variable:
- The variable in PL/SQL must follow some naming rules like other programming languages:
- Must start with a letter. The name of the variable must begin with ASCII letter. The PL/SQL is not case sensitive so it could be either lowercase or uppercase. For example: v_data and V_DATA refer to the same variables.
- The variable_name should not exceed 30 characters.
- You should make your variable easy to read and understand, after the first character, it may be any number, underscore (_) or dollar sign (\$).
- Must not be reserved words

Handling Variables in PL/SQL:

Declare and initialize variables in the declaration section (**You must declare a variable before referencing it in other statements, including other declarations**)

Assign new values to variables in the execution section (**New value assigned replaces existing value**)

Viewing results (By using **DBMS_OUTPUT.PUT_LINE** procedure provided by Oracle)

Ease of maintenance (when using **%TYPE** or **%ROWTYPE** you declare variables basing the declarations on the definitions of database columns)

Declaring and Initializing PL/SQL Variables

Variable declaration syntax:

identifier [CONSTANT] datatype
[NOT NULL] [:= | DEFAULT expr];



WHERE:

identifier	variable name
CONSTANT	constrains the variable to a fixed value, must be initialized datatype
datatype	one of possible data-types for the variable
NOT NULL	constrains the variable to have a value , must be initialized
:= DEFAULT	the way to initialize variables
expr	value of initialization can be literal expression, another variable or an expression involving operators and functions

PL/SQL data types

PL/SQL has two kinds of data types: scalar and composite.

The scalar types are types that store single values such as number, Boolean, character, and datetime whereas the composite types are types that store multiple values, for example array, record and collection.

PL/SQL Scalar Data Types and Subtypes



#	Category & Description
1	Scalar Single values with no internal components, such as a NUMBER, DATE, or BOOLEAN.
2	Large Object (LOB) Pointers to large objects that are stored separately from other data items, such as text, graphic images, video clips, and sound waveforms.
3	Composite Data items that have internal components that can be accessed individually. For example, collections and records.
4	Reference Pointers to other data items.

Types of Variables

PL/SQL variables:

- All SQL
- Scalar
- Composite
- Reference
- Large objects (LOB)

TRUE

25-JAN-01



The soul of the lazy man desires, and has nothing; but the soul of the diligent shall be made rich.

256120.08



Atlanta

TRUE	23-DEC-98	ATLANTA	
------	-----------	---------	---

Declaring and Initializing PL/SQL Variables

Example 1

```
SET SERVEROUTPUT ON
DECLARE
  Myname VARCHAR2(20);
BEGIN DBMS_OUTPUT.PUT_LINE('My name is:
'||Myname);
  Myname := 'John';
  DBMS_OUTPUT.PUT_LINE('My name is: '||Myname);
END;
/
```

My name is:
My name is: John

PL/SQL procedure successfully completed.

Example 2

```
SET SERVEROUTPUT ON
DECLARE
  Myname VARCHAR2(20):= 'John';
BEGIN
  Myname := 'Steven';
  DBMS_OUTPUT.PUT_LINE('My name is: '||Myname);
END;
/
```

My name is: Steven

PL/SQL procedure successfully completed.

Guidelines for Declaring PL/SQL Variables

Avoid using column names as identifiers.

```
DECLARE
  employee_id    NUMBER(6);
BEGIN
  SELECT  employee_id
        INTO      employee_id
        FROM      employees
        WHERE     last_name = 'Kochhar';
END;
/
```

Delimiters in String Literals

In Oracle SQL and PL/SQL, string literals are sequences of characters enclosed in delimiters that define the beginning and end of the string. Understanding the different delimiters Oracle supports is essential for writing and interpreting queries correctly.

1. Single Quotation Marks (') This is the most common delimiter used for string literals.

2. To include a single quote inside the string:

- use two single quotes:
- q' Delimiter Syntax -q'[delimiter text delimiter]'
- You can use any matching pair of delimiters: [], {}, (),
 - <> or even non-paired ones like q'!text!'

```
SET SERVEROUTPUT ON
DECLARE
    event VARCHAR2(15);
BEGIN
    event := q'!Father's day!';
    DBMS_OUTPUT.PUT_LINE('3rd Sunday in June is : '||event);
    event := q'[Mother's day]';
    DBMS_OUTPUT.PUT_LINE('2nd Sunday in May is : '||event);
END;
/
```

3rd Sunday in June is : Father's day
2nd Sunday in May is : Mother's day

PL/SQL procedure successfully completed.

Guidelines for Declaring and Initializing PL/SQL Variables

- Follow naming conventions.
- Use meaningful names for variables.
- Initialize variables designated as NOT NULL and CONSTANT.
- Initialize variables with the assignment operator (:=) or the DEFAULT keyword:

```
Myname VARCHAR2 (20) := 'John' ;
```

```
Myname VARCHAR2 (20) DEFAULT 'John' ;
```

- Declare one identifier per line for better readability and code maintenance.

Guidelines for Declaring PL/SQL Variables

Avoid using column names as identifiers.

```
SET SERVEROUTPUT ON
DECLARE
    employee_id NUMBER(6);
BEGIN
    SELECT employee_id
    INTO     employee_id
    FROM     employees
    WHERE    last_name = 'Chen';
    DBMS_OUTPUT.PUT_LINE('Employee id is :
    '||employee_id);
END;
/
```

The %TYPE Attribute

Is used to declare a variable according to:

- A database column definition
- Another declared variable

Is prefixed with:

- The database table and column
- The name of the declared variable

Declaring Variables with the %TYPE Attribute

Syntax:

```
identifier      table.column_name%TYPE;
```

Examples:

```
...  
  emp_lname      employees.last_name%TYPE;  
  balance        NUMBER(7,2);  
  min_balance    balance%TYPE := 1000;  
...
```

Declaring Boolean Variables

Only the values TRUE, FALSE, and NULL can be assigned to a Boolean variable.

Conditional expressions use logical operators AND, OR, and unary operator NOT to check the variable values.

The variables always yield TRUE, FALSE, or NULL.

Bind variables are:

- Created in the environment
- Also called host variables
- Created with the **VARIABLE** keyword
- Used in SQL statements and PL/SQL blocks
- Accessed even after the PL/SQL block is executed
- Referenced with a preceding colon

```
VARIABLE emp_salary NUMBER
BEGIN
    SELECT salary
    INTO :emp_salary
    WHERE employee_id=120;
END;
/
PRINT emp_salary
```

Substitution Variables

Are used to get user input at run time

Are referenced within a PL/SQL block with a preceding ampersand

Are used to avoid hard coding values that can be obtained at run time

```
VARIABLE emp_salary NUMBER
SET AUTOPRINT ON
DECLARE
empno NUMBER(6) := &empno;
BEGIN
SELECT salary
INTO :emp_salary
FROM employees
WHERE employee_id = empno;
END;
/
```

Bind & Substitution Variables

```
SET VERIFY OFF
VARIABLE emp_salary NUMBER
ACCEPT empno PROMPT 'Please enter a valid employee number: '
SET AUTOPRINT ON

DECLARE
empno NUMBER(6) := &empno;
BEGIN
    SELECT salary INTO :emp_salary
    FROM employees
    WHERE employee_id = empno;
END;
```


Summary & Homework



THEORY

[PL/SQL 101 Part 1-5](#)

PRACTICE

- Review HR Schema
- [PL/SQL Anonymous Blocks](#)

HOMEWORK

- Setup local - HR Schema
- SQL Live
- Create accounts